

DSA Gaming Platform



Session 2023 – 2027

Submitted by:

ESA IRFAN	2023-CS-104
Waleed Sarfaraz	2023-CS-102
Muhammad Owais	2023-CS-131

Supervised by:

Mr. Nazeef ul Haq
Mr. Waseem

Course:

CSC-200 Data Structures and Algorithms

Department of Computer Science

**University of Engineering and Technology
Lahore Pakistan**

Contents

Table of Figures	iv
Table of Tables	v
1.1 Introduction.....	1
1.2 Objectives	1
1.3 Scope.....	1
1.4 Overview	1
1.4.1 Gaming Components	1
1.4.1.1 Stack Related	2
1.4.1.2 Linked List Related.....	5
1.4.1.3 Array Related.....	6
1.4.1.4 Tree Related	10
1.4.1.5 Graph Related	13
1.4.2 Learning Components	16
1.5 Methodology	17
1.5.1 Conceptual Framework.....	17
1.5.2 Gaming Component	17
1.5.3 Learning Component	18
1.5.4 Technical Implementation.....	18
1.5.4.1 Modular Structure	18
1.5.4.2 Frontend Development.....	18
1.5.4.3 Backend Development	18
1.5.4.4 Integration Between Components	19
1.5.4.5 Scalability	19
1.5.4.6 Testing and Debugging	19
1.5.5 Technologies used.....	19
1.5.6 Working Flow	20
1.5.6 Outcomes	20
1.5.7 Organization.....	20
1.5.8 GitHub Link.....	21
1.6 Implementation of Classes.....	21
1.6.1 Card Class (Memory Match).....	21
1.6.2 TicTacToe Class	22

1.6.3 WordSearch Class.....	22
1.6.4 Snake Class	23
1.6.5 FillUpTheBottle Class	23
1.6.6 TowerOfHanoi Class	23
1.6.7 BinarySearchTree Class.....	24
1.7 Test Cases	24
1.7.1 General Test Case Structure.....	24
1.7.2 Test Case for "Fill Up the Bottle" Game	25
1.7.3 Test Case for "Tower of Hanoi" Game	26
1.7.4 Test Case for "Snake Evolution".....	27
1.7.5 Test Case for "Maze Solver".....	27
1.7.6 Test Case for "Traversal Tycoon".....	27
1.7.7 Test Case for "Tic Tac Toe"	28
1.7.8 Test Case for "Balanced Tree Formatter"	29
1.7.8 Test Cases for Quiz Component	30
1.8 Wireframes.....	30
1.8.1 Starting Menus	30
1.8.1.1 Main Page	30
1.8.1.2 Select Game Menu.....	31
1.8.1.3 Quiz Menu	32
1.8.1.4 About Us Menu.....	32
1.8.2 Game Menus	33
1.8.2.1 Memory Match.....	33
1.8.2.2 Word Search.....	33
1.8.2.3 One Way Out	34
1.8.2.4 Maze Solver	34
1.8.3 Restart Menu.....	35
1.9 Conclusion	35
1.10 References.....	35

Table of Figures

Figure 1 Working Flow	20
Figure 2 Directory of Project	21
Figure 3 Test Case 1 (Valid Ball Movement)	25
Figure 4 Test Case 2 (Invalid Ball Movement).....	26
Figure 5 Test Case 3 (Movement of Tile)	26
Figure 6 Test Case 4 (Collision)	27
Figure 7 Test Case 6 (Pre Order)	28
Figure 8 Test Case 7 (Placing X).....	29
Figure 9 Test Case 8 (Insertion in tree)	29
Figure 10 Test case 10 (Valid Answer)	30
Figure 11 Main Menu.....	31
Figure 12 Game Menu	31
Figure 13 Quiz Menu.....	32
Figure 14 About Us Menu	32
Figure 15 Memory Match	33
Figure 16 Word Search.....	33
Figure 17 One Way Out.....	34
Figure 18 Maze Solver.....	34
Figure 19 Restart Menu	35

Table of Tables

Table 1 Fill Up the Bottle.....	4
Table 2 Tower of Hanoi.....	5
Table 3 Snake Evolution.....	6
Table 4 Tic Tac Toe.....	8
Table 5 Memory Match.....	9
Table 6 Word Search.....	10
Table 7 Traversal Tycoon.....	12
Table 8 Balanced Tree Formatter.....	13
Table 9 Maze Solver.....	14
Table 10 One Way Out.....	16
Table 11 Language and Library.....	19

1.1 Introduction

Data Structures and Algorithms (DSA) form the backbone of computer science and software development. Mastering these concepts is essential for solving complex computational problems efficiently and is a fundamental skill for developers and programmers. However, learning DSA can often seem daunting and abstract, particularly for beginners. To bridge this gap, the *DSA Gaming Platform and Learning Area* project was conceived, aiming to combine the engaging nature of gaming with the educational rigor of DSA learning.

This project is a unique blend of an interactive **gaming platform** and a **learning environment**. The gaming platform consists of ten carefully curated games, each designed to reinforce specific DSA concepts such as stacks, queues, linked lists, trees, and graphs. The primary goal of the project is to make DSA learning more accessible and engaging for users of all levels.

1.2 Objectives

- To create an interactive gaming platform that simplifies complex DSA concepts.
- To provide a learning environment with quizzes and challenges to test and reinforce understanding.
- To foster an engaging and motivating learning experience by gamifying educational content.
- To make DSA concepts applicable to real-world scenarios through well-designed games.

1.3 Scope

The project is designed for students, developers, and enthusiasts who want to enhance their DSA knowledge through hands-on learning. The gaming platform offers a diverse range of challenges that cater to both beginners and advanced learners, while the learning platform ensures continuous improvement through structured quizzes and feedback mechanisms.

1.4 Overview

The *DSA Gaming Platform and Learning Area* project is an innovative solution designed to make learning Data Structures and Algorithms (DSA) more interactive, engaging, and practical. By combining a **gaming platform** with a **learning environment**, the project aims to bridge the gap between theoretical knowledge and its real-world applications. Below is an overview of its key components:

1.4.1 Gaming Components

The gaming platform features **ten uniquely designed games**, each addressing a specific concept of DSA. The goal is to provide users with hands-on experience by integrating DSA principles into gameplay mechanics.

1.4.1.1 Stack Related

1.4.1.1.1 Fill Up the Bottle

Concept:

"Fill Up the Bottle" is a logic-based puzzle game where players must organize colored balls into separate bottles. The game represents the concept of stacks by allowing players to move balls based on specific rules. Each bottle must contain balls of only one color in the correct order.

Gameplay Rules

1. **Move Rules:**
 - A ball can only be moved if the target bottle:
 - Is empty.
 - Has its top ball the same color as the ball being moved?
 - A bottle can hold a maximum of four balls.
2. **Gameplay Features:**
 - **Drag-and-Drop Mechanics:** Players can drag and drop balls to move them between bottles.
 - **Visual Cues:** Highlight valid moves to assist the player.
 - **Challenges:**
 - **Timer:** Track how quickly the puzzle is solved.
 - **Move Counter:** Count the number of moves used to solve the puzzle.
 - **Win Condition:** A success message is displayed once all bottles are correctly sorted.

Learning Outcomes

- **Understanding Stack Operations:**
 - Push: Adding a ball to a bottle.
 - Pop: Removing a ball from a bottle.
- **Logical Thinking:**
 - Manage constraints effectively.
 - Use empty bottles as temporary storage to achieve the goal.
- **Visualization of LIFO Principle:**
 - The last ball added to a bottle is the first to be removed (Last In, First Out).

Code Structure

frontend.py

Handles game visuals and user interaction, implemented with **Pygame**.

- **Major Functionalities:**
 - **Visual Representation:**
 - Bottles are drawn using rounded rectangles.
 - Balls are drawn as circles and color-coded.
 - **Interactivity:**
 - Mouse clicks identify selected bottles and determine valid moves.
 - Dragging balls is animated for realism.

- **Animations:**
 - Smooth ball movement between bottles.
 - Bottles and balls are redrawn to reflect game progress.

Key Methods:

- `draw_bottles`: Draws bottles and balls on the screen, highlights selected bottle.
- `animate_ball_upward`: Simulates a ball being picked up from a bottle.
- `move_ball_to_target`: Handles ball movement logic and updates the display.
- `reset_selected_bottle`: Resets selection after a move or invalid action.

Game Flow in `main()`:

1. Initialize game objects and start the main loop.
2. On user interaction:
 - Check bottle selection.
 - Validate moves and handle logic accordingly.
3. Redraw the screen and check for completion.
4. Display a win screen when the game is complete.

`logic.py`

Defines game mechanics and manages the state.

- **Initialization:**
 - Generates bottles with random distributions of colored balls.
 - Adds two empty bottles for gameplay flexibility.
- **Validation Rules:**
 - Ensures moves adhere to the rules (e.g., balls match colors, bottles have space).
- **Game State:**
 - Tracks bottle contents.
 - Detects when the game is complete by ensuring all bottles are sorted with a single color.

Key Methods:

- `_initialize_game`: Randomly fills bottles with balls.
- `is_valid_move`: Validates whether a ball can move from a source bottle to a target bottle.
- `is_game_complete`: Checks if all bottles are sorted correctly.
- `make_move`: Executes a valid move and tracks the number of moves made.

Game	Fill Up the Bottle
Data Structure used	Stack
Concept used related with Data structure	LIFO (Last In, First Out) principle; stack operations (push/pop).
Purpose/Use Case	Organize balls in bottles by color using stack operations.

Advantages	- Teaches practical use of stack operations. - Encourages problem-solving and logical thinking. - Interactive gameplay for hands-on learning.
-------------------	---

Table 1 Fill Up the Bottle

1.4.1.1.1 Tower of Hanoi

Concept

The **Tower of Hanoi** is a classic problem-solving game where players must transfer a stack of disks from one rod (peg) to another while adhering to a set of rules. This project implements the game using Python and Pygame, adding visual and auditory feedback for a rich gaming experience.

Gameplay Rules

- Only one disk can be moved at a time.
- A larger disk cannot be placed on top of a smaller disk.
- Players must transfer all disks to a target rod while following the rules.
- The game is completed when all disks are stacked on the target rod in the correct order (smallest on top, largest on bottom).
- Visual and sound effects provide feedback on valid and invalid moves.

Learning Outcomes

- **Understanding Stack Operations:**
 - Push: Adding a disk to a rod.
 - Pop: Removing a disk from a rod.
- **Logical Thinking:**
 - Learn and apply recursion principles to solve logical puzzles.
- **Visualization of LIFO Principle:**
 - Observe how “Last In, First Out” works in real-time.

Code Structure

The project is structured into two files and uses **assets** for sounds and visuals:

Logic (logic.py)

This file handles the backend logic of the Tower of Hanoi game:

- **Initialization:** Sets up the rods and disks.
- **Validation:** Checks if a move is valid based on the game rules.
- **Completion Check:** Determines if the game is successfully completed.

Frontend (frontend.py)

This file manages the visual and interactive components of the game:

- **Graphics:** Uses Pygame to render rods, disks, and background visuals.

- **Interactions:** Handles user input (mouse clicks) for selecting and moving disks.
- **Animations:** Smoothly animates the movement of disks between rods.
- **Sound Effects:** Plays sounds for valid moves and invalid actions

Game	Tower of Hanoi
Data Structure used	Stack
Concept used related with Data structure	LIFO (Last In, First Out) principle; stack operations (push/pop).
Purpose/Use Case	Store disks on each peg (stack) in decreasing order of size (from bottom to top).
Advantages	- Simple and efficient for managing disk movements. - Supports recursion for solving the problem. - Ideal for scenarios like the Tower of Hanoi where only the top disk of each stack is accessible at any given time.

Table 2 Tower of Hanoi

1.4.1.2 Linked List Related

1.4.1.2.1 Snake Evolution

Concept

The **Snake Evolution** game builds upon the classic Snake game, introducing a twist where if the snake's head collides with any part of its body, the collided node and all subsequent nodes are removed from the snake's body. This creates a new layer of strategy for players, as they must carefully navigate their path to avoid collisions and maintain the snake's length.

Gameplay Rules

- **Snake Growth:** The snake grows longer with each food item collected. The growth is represented as a **dynamically linked list** where each node corresponds to a segment of the snake.
- **Collisions:** When the snake's head collides with its body, the segment at the collision point and all subsequent nodes are deleted.
- **Game Over:** If only one segment remains, the game ends.
- **Strategy:** Players must balance growth with strategic planning to avoid reducing the snake's length.
- **Objective:** The goal is to survive as long as possible by collecting items to grow the snake while avoiding collisions that could shorten the snake or end the game.

Learning Outcomes

- **Dynamic Data Structures:** Students will learn how to implement a **linked list** for dynamically managing the snake's body. This demonstrates how data structures can be used to represent real-world concepts like a snake.
- **Pointer Manipulation:** The game emphasizes understanding pointer manipulation as nodes are dynamically added or removed from the linked list.
- **Logical Thinking & Strategy:** The game encourages players to think strategically about how to navigate the snake's path, reinforcing decision-making skills.

Code Structure

The project is structured into two files and uses **assets** for sounds and visuals:

Logic (logic.py)

- **Snake movement and growth:** Manages snake's movement, using linked list data structures to update its position, and handles snake growth when it eats food.
- **Collision detection:** Checks for collisions with walls, the snake's body, or food, ending the game on certain collisions.
- **Food generation:** Randomly places food on the screen and updates when the snake eats it.
- **Game logic:** Controls the core gameplay, including snake growth, self-collision detection, and score tracking.
- **Efficient data handling:** Uses linked list structure for dynamic and efficient management of the snake's body and movement.

Frontend (frontend.py)

- **Handles game visuals:** Renders the snake, food, and game environment on the screen using Pygame's drawing functions.
- **User input management:** Captures user input for snake movement, pausing, and exiting the game.
- **Game state display:** Shows the score, game over, and pause screens, updating them dynamically based on the game's progress.
- **Game Over and Pause Mechanism:** Manages when the game should pause or end, displaying appropriate messages on the screen.
- **Enhances player experience:** Uses sound effects and background music to make the game more engaging.

Game	Snake Evolution
Data Structure used	Linked List
Concept used related with Data structure	A linked list allows for flexible addition and removal of snake segments during the game. Each segment is a node, which is dynamically linked to the next segment.
Purpose/Use Case	Used for representing the snake's body.
Advantages	-- Dynamic memory usage; no predefined size. - Efficient insertions and deletions at any position

Table 3 Snake Evolution

1.4.1.3 Array Related

1.4.1.3.1 Tic Tac Toe

Concept

Tic Tac Toe is a classic two-player strategy game played on a 3x3 grid. The goal is to get three of your marks (either "X" or "O") in a row, either horizontally, vertically, or diagonally, before your opponent does.

Gameplay Rules

- **Grid Setup:** The game is played on a 3x3 grid, with 9 cells.
- **Players:** There are two players, one using the "X" symbol and the other using the "O" symbol.
- **Turn-based Play:** Players take turns marking one empty cell in the grid with their respective symbol.
- **Win Condition:** A player wins if they align three of their marks in a row, either horizontally, vertically, or diagonally.
- **Draw Condition:** If all cells are filled and no player has won, the game ends in a draw.
- **Game Reset:** The game can be restarted after a win or draw, with the board reset to its original state.

Learning Outcomes

- **Array Manipulation:** Represent the Tic Tac Toe grid as a 2D array (or list of lists in Python) to model the game state.
- **Win-Checking Algorithms:** Implement algorithms to check for a win by inspecting rows, columns, and diagonals.
- **Turn-based Logic:** Learn how to implement turn-based game logic to alternate player turns.
- **User Input:** Dynamically handle user input to place symbols in the grid.
- **Game Flow Management:** Implement a basic game loop to manage gameplay and check for win conditions.

Code Structure

The project is structured into two files and uses **assets** for sounds and visuals:

Logic (logic.py)

- **Grid Representation:** The game grid is represented as a 2D array (3x3 grid), where each cell holds a value representing either 'X', 'O', or is empty.
- **Turn-based Gameplay:** Players take turns to place their respective symbol (X or O) in an available cell of the grid.
- **Win-Checking Algorithm:** After each move, the game checks for winning conditions by evaluating rows, columns, and diagonals for three consecutive identical marks ('X' or 'O').
- **Draw Detection:** The game also checks if all cells are filled and no player has won, resulting in a draw.
- **Efficient State Updates:** The game ensures that each turn correctly updates the grid and checks for the end conditions (win, draw, or ongoing game).

Frontend (frontend.py)

- **Pygame Setup:**
 - Initializes a Pygame window to render the game.
 - Sets up the screen dimensions and the window title.
- **Rendering the Grid:**
 - Draws the 3x3 Tic Tac Toe grid on the screen.
 - Renders the 'X' and 'O' symbols in the grid cells during each player's turn.

- **Handling User Input:**
 - Captures mouse events to let players click on available cells.
 - Ensures that moves are only made in empty cells.
- **Win, Draw, and Ongoing Status Display:**
 - Displays messages like "Player X wins", "Player O wins", or "Draw" based on the game's outcome.
 - Updates the screen after each move to reflect the latest game status.
- **Game Restart Mechanism:**
 - Provides an option to restart the game once it ends (either win or draw).
- **User Interface Enhancements:**
 - Uses colors and fonts to make the game visually appealing.
 - Ensures a user-friendly experience by updating the screen and displaying relevant messages at the appropriate times.

Game	Tic Tac Toe
Data Structure used	1D or 2D array
Concept used related with Data structure	Learn how to insert in 2d array and knows the working of 2D array
Purpose/Use Case	Store the 3x3 game board, where each element represents a cell in the grid
Advantages	Simple to access and modify specific elements.

Table 4 Tic Tac Toe

1.4.1.3.2 Memory Match

Concept

A memory-based matching game where players flip cards on a grid to find pairs. The goal is to match all pairs by recalling the positions of cards.

Gameplay Rules

- The grid contains pairs of cards with hidden symbols or images.
- Players click on two cards to flip and reveal their contents:
 - If the cards match, they remain visible.
 - If they do not match, they are flipped back after a brief delay.
- The game ends when all pairs are matched.

Learning Outcomes

- Practice working with 2D arrays for storing and accessing data.
- Develop logic to compare elements (cards) effectively.
- Enhance event-handling skills, such as mouse click detection.
- Improve understanding of turn-based gameplay design and state management.

Code Structure

The project is structured into two files and uses **assets** for sounds and visuals:

Logic (logic.py)

- Generates a shuffled grid of card pairs using a 2D array.
- Represents each card with attributes like position, symbol, and flipped/matched status.
- Implements logic to determine if two flipped cards match.
- Handles card management and tracking flipped or matched states.

Frontend (frontend.py)

- Initializes the Pygame window and visual elements like the card grid, fonts, and images.
- Draws the cards and manages their flipping and matching states.
- Plays sound effects for flipping and successful matches.
- Handles user interactions, such as detecting clicks and updating the game state.

Game	Memory Match
Data Structure used	1D or 2D array
Concept used related with Data structure	Grid Representation
Purpose/Use Case	Stores the positions and symbols of the cards.
Advantages	Efficiently manages card positions and enables systematic grid traversal.

Table 5 Memory Match

1.4.1.3.3 Word Search

Concept

Players engage in a Word Search game where they find hidden words in a grid filled with random letters. Words are placed horizontally, vertically, or diagonally. The goal is to locate all the hidden words from the given word list within a set time.

Gameplay Rules

- A grid is displayed with random letters, containing hidden words.
- Players select letters in sequence to form words.
- A list of words to find is provided.
- Words can be hidden horizontally, vertically, or diagonally.
- Time and found words are tracked during gameplay.

Learning Outcomes

- Practice working with 2D arrays and nested loops.
- Understand and implement algorithms for searching patterns in grids.
- Learn logic for diagonal traversal in arrays.
- Gain experience with validating user input dynamically.
- Develop proficiency in integrating logic with Pygame for user interaction

Code Structure

The project is structured into two files and uses **assets** for sounds and visuals:

Logic (logic.py)

- Generates a shuffled grid of card pairs using a 2D array.
- Represents each card with attributes like position, symbol, and flipped/matched status.
- Implements logic to determine if two flipped cards match.
- Handles card management and tracking flipped or matched states.

Frontend (frontend.py)

- **Grid Visualization:** Creates a grid of letters and highlights selected or found words using colors.
- **Word List Rendering:** Displays the list of words to be found, dimming words once found.
- **Mouse Interaction:** Maps player mouse clicks to grid cells for word selection.
- **UI Design:** Centers the grid on the screen and includes visual aesthetics like borders and fonts

Game	Word Search
Data Structure used	1D or 2D array
Concept used related with Data structure	Grid Representation
Purpose/Use Case	Stores the layout of the grid for visualization and word searches
Advantages	Easy to iterate and validate selections. Simplifies traversal for word placement and selection

Table 6 Word Search

1.4.1.4 Tree Related

1.4.1.4.1 Traversal Tycoon

Concept

The **Traversal Tycoon** visualizes various tree traversal algorithms (pre-order, in-order, and post-order) on a randomly generated tree. Players aim to correctly identify the nodes visited in a specific traversal order by clicking on them.

Gameplay Rules

- **Game Start:** The user chooses a desired tree traversal type (pre-order, in-order, or post-order) from a menu driven by `supportive_frontend.py`.

- **Tree Generation and Layout:** logic.py generates a random tree with TreeNode objects and calculates node positions on the screen using get_tree_layout.
- **Visualization:** frontend.py draws the tree structure with circles representing nodes and lines connecting parent-child nodes. Nodes are initially colored blue.
- **Traversal Display:** The selected traversal order is displayed at the bottom of the screen.
- **Player Interaction:** Players click on nodes.
 - **Correct Click:** If the clicked node matches the first node in the traversal order, it turns green, the order is updated, and a sound effect plays (button_click.mp3).
 - **Incorrect Click:** If the clicked node is not the first in the order, it turns red and a different sound effect plays (incorrect_click.mp3). The incorrect node turns back to blue after a short delay.
- **Game Completion:** The game continues until all nodes in the traversal order are selected correctly.
- **Restart Option:** Users can likely restart the game (exact implementation details are not provided in the code, but it can be added using techniques like buttons or keyboard shortcuts).

Learning Outcomes

- **Visualization:** Players gain a visual understanding of how tree traversals work by seeing the order in which nodes are visited.
- **Interactive Learning:** The game provides an interactive way to practice and reinforce knowledge of tree traversal algorithms.
- **Feedback:** Immediate feedback based on player clicks helps them learn from mistakes and improve accuracy.

Code Structure

The project is structured into two files and uses **assets** for sounds and visuals:

Logic (logic.py)

- Defines a TreeNode class to represent nodes in the tree, each with a value and a list of children.
- Creates a function generate_random_tree to build a random tree with a specified number of nodes.
- Implements functions for different tree traversals:
 - preorder_traversal
 - inorder_traversal
 - postorder_traversal
- Offers a function get_traversals to return a dictionary containing the results of all three traversals.

Frontend (frontend.py)

- Handles user interaction and visualizes the game.
- Calculates node positions on the screen based on their depth in the tree.
- Draws nodes as circles and connects them with lines to represent the tree structure.
- Tracks node colors based on user interaction (correct, incorrect, unselected).

- Updates the display of the selected traversal order as players choose nodes.
- Plays sound effects for correct and incorrect clicks to provide feedback

Game	Traversal Tycoon
Data Structure used	Trees
Concept used related with Data structure	Learn how to get the in order, preorder and post order of the tree
Purpose/Use Case	Store the 13 to 15 nodes tree.
Advantages	Enables efficient hierarchical organization of data

Table 7 Traversal Tycoon

1.4.1.4.1 Balanced Tree Formatter

Concept

The **Balanced Tree Formatter** is designed to teach players about binary search trees (BSTs) and AVL trees through an interactive experience. Players start with a balanced BST and a set of extra nodes. The goal is to insert these extra nodes into the BST while maintaining its balance.

Gameplay Rules

- **Initial State:** A balanced BST is displayed on the screen, along with a set of extra nodes.
- **Node Selection:** Players can select an extra node by clicking on it.
- **Node Placement:** Players can use keyboard keys (R, L, Up) to navigate the BST and insert the selected extra node at a suitable position.
- **Game Completion:** The game ends when all extra nodes are inserted into the BST. The player wins if the final tree is balanced; otherwise, they lose.

Learning Outcomes

- **Understanding BST Structure:** Players learn about the hierarchical structure of binary search trees, where nodes are arranged in a specific order.
- **BST Insertion:** Players practice inserting nodes into a BST, which involves comparing values and finding the correct position.
- **AVL Tree Balancing:** Players understand the concept of AVL trees, which are self-balancing BSTs. They learn how rotations are used to maintain balance.
- **Problem-Solving and Critical Thinking:** Players need to strategize and think critically about where to insert nodes to minimize imbalances in the tree.

Code Structure

The project is structured into two files and uses **assets** for sounds and visuals:

Logic (logic.py)

- **Node Class:** Defines the structure of a node in the BST, including key, left/right child references, and height.
- **BinarySearchTree Class:** Implements core BST operations like insertion, deletion, and searching.
- **AVL Tree Balancing:** Includes methods for left and right rotations to maintain balance in the BST.
- **Height Calculation:** Calculates the height of a node, which is crucial for AVL balancing.
- **Win Condition:** Defines the criteria for winning the game, often based on achieving a specific tree height or structure.

Frontend (frontend.py)

- **Game Initialization:** Initializes the Pygame window, sets up the screen, and loads necessary assets.
- **Visual Representation:** Draws the BST on the screen, including nodes, edges, and node values.
- **User Interaction:** Handles user input from the keyboard and mouse, allowing players to navigate the tree and insert nodes.
- **Game Loop:** Maintains the game loop, updating the screen, handling user input, and checking for game-over conditions.
- **Feedback and Messages:** Provides visual and audio feedback to the player, such as highlighting nodes, displaying messages, and playing sound effects.

Game	Balanced Tree Formatter
Data Structure used	Trees
Concept used related with Data structure	Binary Search Tree, AVL Tree
Purpose/Use Case	Implements BST operations, AVL balancing, and game logic
Advantages	Provides the foundation for the game's mechanics and decision-making.

Table 8 Balanced Tree Formatter

1.4.1.5 Graph Related

1.4.1.5.1 Maze Solver

Concept

A maze solver game is a puzzle where a player guides a character (often a mouse or robot) through a maze from a starting point to a goal. The game uses algorithms like Breadth-First Search (BFS) and Depth-First Search (DFS) to solve the maze automatically or to assist the player in finding the solution.

Gameplay Rules

- **Maze Generation:** The game generates a random maze or uses a predefined maze.

- **Player Control:** Players can either control the character directly or let the algorithm solve the maze automatically.
- **Algorithm Selection:** Players can choose between BFS and DFS to solve the maze.
- **Maze Visualization:** The game visualizes the maze, the character's movement, and the path found by the algorithm.
- **Win Condition:** The game ends when the character reaches the goal.

Learning Outcomes

- **Understanding Search Algorithms:** Players can learn about BFS and DFS, how they work, and their differences.
- **Problem-Solving Skills:** Players can practice problem-solving skills by trying to solve the maze manually or by analyzing the algorithm's solution.
- **Visualizing Algorithms:** The game provides a visual representation of the algorithms, making it easier to understand their step-by-step execution.

Code Structure

The project is structured into two files and uses **assets** for sounds and visuals:

Logic (logic.py)

- **Maze Representation:** A 2D array is used to represent the maze, with each cell indicating a wall, path, or goal.
- **BFS Implementation:** A queue-based algorithm to explore the maze level by level, ensuring the shortest path.
- **DFS Implementation:** A stack-based algorithm to explore the maze depth-first, potentially finding a solution faster but not necessarily the shortest path.

Frontend (frontend.py)

- **Game Initialization:** Initializes the Pygame window, sets up the screen, and loads necessary assets.
- **Maze Visualization:** Draws the maze on the screen, including walls, paths, and the character.
- **Algorithm Visualization:** Visualizes the algorithm's progress by highlighting visited cells and the path found.
- **User Interaction:** Handles user input, such as starting the algorithm or controlling the character.

Game	Maze Solver
Data Structure used	Graph
Concept used related with Data structure	Learn how to implement the BFS and DFS for searching path
Purpose/Use Case	Implement the maze generation, algorithm implementation.
Advantages	Provide the foundation of the game's functionality

Table 9 Maze Solver

1.4.1.5.2 One Way Out

Concept

The Euler Path game is a puzzle where the player is presented with a graph (represented as nodes and edges). The goal is to find a Eulerian path, a path that visits every edge exactly once. Players can click on nodes to traverse the graph. The game uses graph theory concepts and algorithms to determine if an Eulerian path exists and to guide the player towards a solution.

Gameplay Rules

- **Node Selection:** Players click on nodes to select them.
- **Path Formation:** A path is formed by connecting selected nodes with edges.
- **Win Condition:** The game is won when all edges have been traversed exactly once.
- **Lose Condition:** The game is lost if the player gets stuck and cannot find a valid path to continue.

Learning Outcomes

- **Graph Theory Concepts:** Players learn about graphs, nodes, edges, and the concept of Eulerian paths.
- **Problem-Solving Skills:** Players practice problem-solving by analyzing the graph and planning their moves.
- **Algorithmic Thinking:** Players can learn about algorithms for finding Eulerian paths, such as Fleury's algorithm.

Code Structure

The project is structured into two files and uses **assets** for sounds and visuals:

Logic (logic.py)

- **Graph Representation:** The graph is represented using a list of nodes and a list of edges.
- **Win/Lose Conditions:** Functions to check if all edges have been used (win) or if no valid moves are left (lose).
- **Edge Usage Tracking:** A set is used to keep track of used edges.
- **Node Selection and Path Formation:** Handles the logic of selecting nodes and updating the used edges.

Frontend (frontend.py)

- **Game Initialization:** Sets up the Pygame window, defines colors, and initializes variables for nodes, edges, and game state.
- **Visualization:** Draws the graph on the screen, including nodes as circles and edges as lines.
- **User Interaction:** Handles mouse clicks to select nodes and update the game state.
- **Win/Lose Conditions:** Checks the win/lose conditions based on the logic provided by logic.py.
- **Feedback and Messages:** Displays win/lose messages and visualizes the path taken by the player.

Game	One Way Out
Data Structure used	Graph
Concept used related with Data structure	Learn how to implement the Flurry Algorithm for learning
Purpose/Use Case	Graph theory, Eulerian path
Advantages	Implements the graph structure, win/lose conditions, and edge usage tracking.

Table 10 One Way Out

1.4.2 Learning Components

The learning component is an interactive quiz-based system designed to test and strengthen the user's theoretical understanding of DSA concepts.

Conceptual Framework

- **Objective:** Provide users with structured MCQs (Multiple Choice Questions) to assess and reinforce their theoretical knowledge.
- **Topics Covered:**
 - Core DSA concepts such as arrays, linked lists, stacks, queues, trees, and graphs.
 - Algorithmic techniques like sorting, searching, and recursion.

Quiz Design

The quizzes are structured to complement the gameplay experience:

- **Question Bank:**
 - A diverse collection of MCQs categorized by difficulty levels (beginner, intermediate, advanced).
 - Questions are based on real-world applications and problem-solving scenarios.
- **Types of Questions:**
 - Conceptual: E.g., *"What is the time complexity of Bubble Sort?"*
 - Practical: E.g., *"What is the output of the following code snippet?"*
 - Problem-Solving: E.g., *"Which data structure is best suited for this task?"*

Learning Outcomes

- **Reinforce DSA Knowledge:** Strengthen understanding of key concepts and algorithms.
- **Bridge Theory and Practice:** Align theoretical questions with gameplay mechanics for a holistic learning experience.
- **Encourage Critical Thinking:** Challenge users to think analytically and solve problems effectively.

Technical Implementation

- **Frontend:**
 - **UI Design:** Simple and user-friendly interfaces with:

- Question display areas.
 - Multiple-choice options (clickable buttons).
 - Feedback mechanisms for answers (e.g., "Correct" or "Incorrect" with explanations).
- **Timer:** An optional timer for each question to add a challenge.
- **Score Display:** Real-time scoring and progress tracking.
- **Backend:**
 - **Database of Questions:** Store questions, options, and correct answers in a structured format.
 - **Question Selection Logic:**
 - Randomized selection to ensure variability.
 - Difficulty-based progression.
 - **Answer Validation:**
 - Check user responses against the correct answers.
 - Provide instant feedback with explanations where needed.

5. Interactive Features

- **Progress Tracking:**
 - Track user performance and provide insights (e.g., "You scored 8/10").
 - Suggest areas for improvement based on incorrect answers.
- **Adaptive Quizzes:**
 - Adjust question difficulty based on user performance.
 - Offer hints for challenging questions to enhance learning.
- **Gamification Elements:**
 - Award points or badges for correct answers to encourage engagement.
 - Leaderboards to foster a competitive spirit among users

1.5 Methodology

The "DSA Gaming Platform and Learning Area" employs a dual-component system—**Gaming** and **Learning**—to provide a comprehensive and engaging approach to mastering Data Structures and Algorithms (DSA). Below is an overview of the methodology:

1.5.1 Conceptual Framework

- **Objective:** Combine gamified practical learning with theoretical reinforcement to simplify DSA concepts for students, developers, and enthusiasts.
- **Topics Covered:** Focuses on stacks, linked lists, arrays, trees, graphs, and algorithms like sorting, searching, and recursion.

1.5.2 Gaming Component

The gaming component introduces DSA concepts interactively:

- Features games such as **Fill Up the Bottle** (stacks), **Maze Solver** (graphs), and **Traversal Tycoon** (trees).

- Games demonstrate real-world applications of DSA concepts like LIFO, graph traversals, and recursive algorithms.
- Frontend utilizes **Pygame** for visual representation and user interaction, while backend logic implements game mechanics like move validation and game state updates.

1.5.3 Learning Component

The learning component complements gaming with structured quizzes:

- MCQ-based quizzes assess users on DSA topics such as time complexity, data structure properties, and algorithms.
- Includes adaptive features like difficulty progression, instant feedback, and performance tracking.
- Quizzes are integrated with the gaming component, unlocking advanced levels or features upon completion.

1.5.4 Technical Implementation

The technical implementation is the backbone of the platform, ensuring seamless functionality and scalability.

1.5.4.1 Modular Structure

- **Component Separation:**
 - **Games:** Organized by DSA concepts (e.g., stack-related games, tree-related games) for modular development.
 - **Quizzes:** Stored as independent modules, allowing easy addition or modification of question sets.

1.5.4.2 Frontend Development

- Built using **Pygame** for consistent visual representation and interactive features:
 - **Game Graphics:** Visualize data structures like trees, graphs, and arrays with intuitive representations (e.g., nodes, edges, grids).
 - **User Interaction:**
 - Drag-and-drop mechanics for games like "Fill Up the Bottle."
 - Mouse and keyboard inputs for quiz responses and navigating game elements
 - **Dynamic Feedback:**
 - Highlight valid moves, display results (e.g., "Correct Answer" or "Win"), and provide animations for user actions.
 - **Timers and Progress Bars:** Enhance gameplay and quiz engagement by tracking time and progress visually.

1.5.4.3 Backend Development

- Written in Python to integrate game mechanics, quiz logic, and DSA operations:
 - **Game Logic:**
 - Implemented algorithms like BFS, DFS, and recursive tree traversals for games.
 - Validates game moves (e.g., stack push/pop, valid graph traversal).

- **Quiz Logic:**
 - Randomly selects questions from a database (e.g., JSON, SQL) based on difficulty level.
 - Tracks user scores and performance metrics for adaptive quizzes.
- **Data Structure Models:**
 - Use data structures such as stacks, linked lists, and graphs to mimic real-world scenarios in games.
 - Ensure optimized memory usage and execution for dynamic gameplay.

1.5.4.4 Integration Between Components

- **Shared Learning Flow:**
 - Games highlight theoretical gaps, encouraging users to attempt quizzes.
 - Quizzes unlock new games or levels to ensure users master concepts before progressing.
- **Cross-Component Feedback:**
 - Learning outcomes from quizzes influence in-game hints and tutorials.
 - Game scores can affect quiz difficulty or focus areas.

1.5.4.5 Scalability

- **Adding New Features:**
 - Modular design allows the easy addition of new games, algorithms, and quizzes without disrupting existing functionalities.
- **Extensible Logic:**
 - Core libraries for data structures and algorithms can be reused across new games or quiz modules.
- **Platform Compatibility:**
 - Designed for deployment on multiple operating systems (Windows, macOS, Linux).

1.5.4.6 Testing and Debugging

- **Unit Testing:**
 - Each game and quiz component are independently tested to validate logic correctness.
 - Data structure manipulations (e.g., stack operations, graph traversal) are verified against expected outcomes.
- **Integration Testing:**
 - Ensure seamless interaction between gaming and learning components.
 - Validate performance under varying user inputs and scenarios.

1.5.5 Technologies used

For the completion of the Project we use following languages with the updated path.

Language	Python
Version	3.13.1
Library	Pygame
Version	2.5.0

Table 11 Language and Library

1.5.6 Working Flow

The Project is working in the below flow :

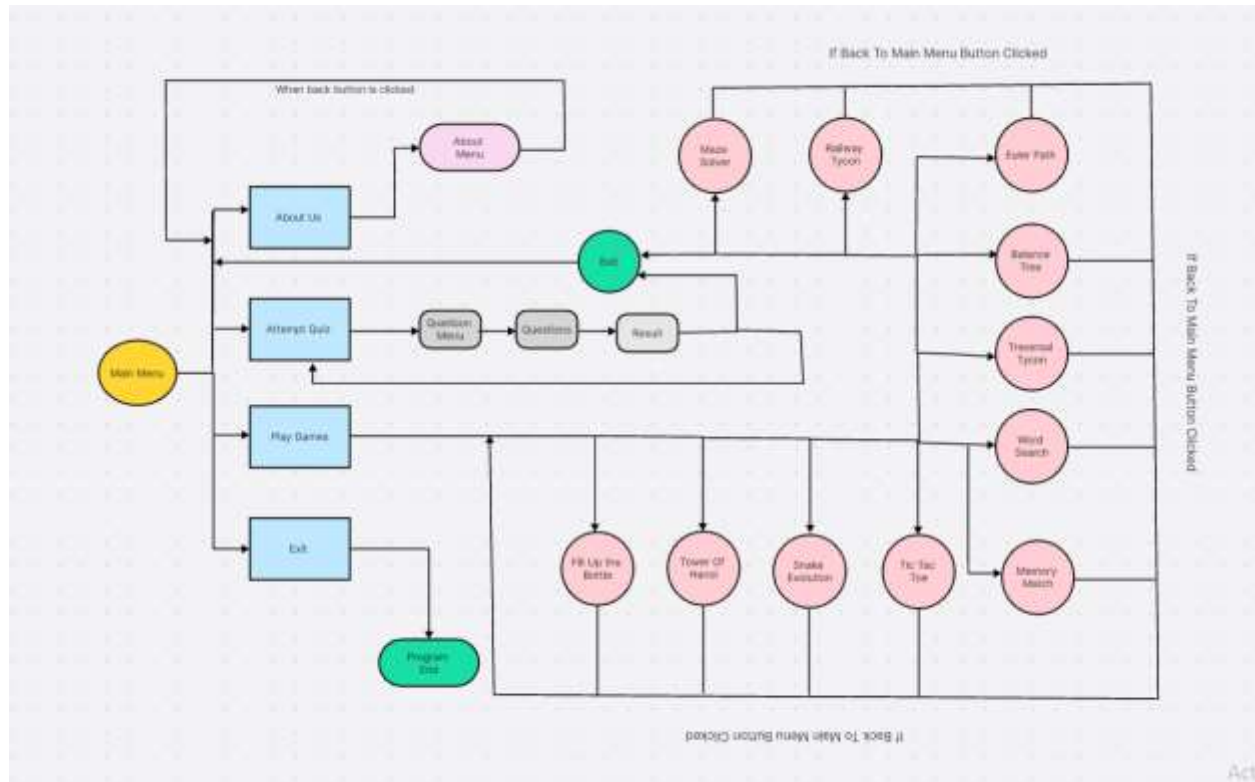


Figure 1 Working Flow

1.5.6 Outcomes

This methodology ensures:

1. **Hands-On Practice:** Games provide an interactive way to apply DSA concepts.
2. **Theoretical Reinforcement:** Quizzes help solidify knowledge through structured assessments.
3. **Scalable Development:** Modular architecture supports future expansion and feature updates.
4. **Engaging Learning:** Gamification and intuitive design keep users motivated and focused.

1.5.7 Organization

- Logic files (logic.py) handle backend mechanics for games and quizzes.
- Frontend files (frontend.py) manage visuals, animations, and interactions.
- Assets such as images, sounds, and fonts are stored in dedicated directories.

```

FINAL_PROJECT/
├── Array_Games/
│   ├── Memory_Match/
│   │   ├── Assets/
│   │   ├── frontend.py
│   │   ├── logic.py
│   │   └── fix_fix_fix/
│   ├── Tic_Tac_Toe/
│   │   ├── Assets/
│   │   ├── frontend.py
│   │   ├── logic.py
│   │   └── Word_Search/
│   │       ├── Assets/
│   │       ├── frontend.py
│   │       ├── logic.py
│   │       └── main.py
│   └── Graph_Games/
│       ├── Maze_Game/
│       │   ├── clock.py
│       │   ├── game.py
│       │   ├── main.py
│       │   ├── maze.py
│       │   ├── player.py
│       │   ├── Assets/
│       │   ├── one_way_gate/
│       │   │   ├── frontend.py
│       │   │   ├── logic.py
│       │   └── railway_tycoon/
│       │       ├── frontend.py
│       │       └── logic.py
│       ├── Linked_List_Games/
│       │   ├── Assets/
│       │   ├── frontend.py
│       │   ├── logic.py
│       │   └── vscode/
│       ├── MainPage/
│       │   ├── Assets/
│       │   └── MainPage.py
│       ├── Quiz_Menu/
│       │   ├── Assets/
│       │   ├── quiz.py
│       │   └── question.py
│       └── Stack_Games/
│           ├── rillingTheBottle/
│           │   ├── Assets/
│           │   ├── frontend.py
│           │   ├── logic.py
│           │   └── UserOfMain/
│           │       ├── Assets/
│           │       ├── frontend.py
│           │       └── logic.py
│           └── Tree_Games/
│               ├── Balanced_Tree/
│               │   ├── Assets/
│               │   ├── frontend.py
│               │   └── logic.py
│               └── Traversal_Tycoon/
│                   ├── Assets/
│                   ├── mainfrontend.py
│                   ├── logic.py
│                   └── supportivefrontend.py
├── README.md
└── report.pdf

```

Figure 2 Directory of Project

1.5.8 GitHub Link

For complete information about project and source code check it

<https://github.com/esairfan/DSA-Gaming-Platform.git>

Hope that you can't face any difficulty in it.

1.6 Implementation of Classes

1.6.1 Card Class (Memory Match)

Introduction

The Card class is a fundamental building block of the **Memory Match Game**, representing each individual card in the game grid. It is responsible for handling the visual representation of cards, their state (flipped or unflipped), and interactions with the user.

Purpose

The primary purpose of the Card class is to encapsulate the behavior and attributes of each card. It manages:

- The card's visual appearance (e.g., displaying the symbol or card back).
- The state of the card, such as whether it is flipped or matched.
- User interactions, allowing the game to detect clicks and respond accordingly.

1.6.2 TicTacToe Class

Introduction

The TicTacToe class is designed to implement the core functionality of the classic Tic Tac Toe game. It manages the game grid, tracks the current player's turn, determines the winner, and ensures the game state is updated correctly after each move. This class provides the necessary logic to facilitate gameplay for two players.

Purpose

The purpose of the TicTacToe class is to:

- **Manage the Game Grid:** Maintain the current state of the game board as a 2D array, with each cell representing a move made by the players.
- **Handle Player Turns:** Alternate turns between players ("X" and "O") and validate their moves.
- **Check Game Outcomes:** Determine if a player has won, if the game has ended in a draw, or if the game should continue.
- **Provide a Reset Mechanism:** Reset the game state to allow for multiple rounds of gameplay without restarting the application.
- **Promote Modular Design:** Encapsulate all game-related logic into a single, reusable class for easier integration into a graphical or console-based interface.

1.6.3 WordSearch Class

Introduction

The WordSearch class is a game logic implementation for creating and managing a word search puzzle. It generates a grid filled with random letters, places a predefined set of words within the grid in a randomized fashion, and validates selected words against the hidden word list.

Purpose

The WordSearch class is designed to:

1. **Generate Word Search Grid:** Dynamically create a square grid of a specified size and populate it with words randomly selected from a predefined word pool.
2. **Place Words in Grid:** Randomly position words horizontally or vertically, ensuring no overlaps except for matching letters.
3. **Validate User Input:** Check if a sequence of selected letters matches any of the hidden words.

4. **Enhance Puzzle Complexity:** Fill unused grid spaces with random letters to create a visually complete and challenging puzzle.
5. **Provide Modular Word Management:** Allow retrieval of the current grid and the hidden word list for display or interaction purposes.

1.6.4 Snake Class

Introduction

The Snake class represents the snake in the game, implemented as a linked list where each node corresponds to a segment of the snake's body. This class manages the movement, growth, and collision detection of the snake, providing the core functionality for the game's snake mechanics.

Purpose

The purpose of the Snake class is to:

- **Control Snake Movement:** Update the snake's position on the grid based on its direction.
- **Handle Growth:** Add new segments to the snake's body as it consumes food.
- **Detect Collisions:** Check if the snake collides with itself, resulting in a game-over condition.
- **Track Snake State:** Manage the state of the snake, including whether it has one segment or more after collision detection.

1.6.5 FillUpTheBottle Class

Introduction

The FillUpTheBottle class implements the logic for the "Fill Up the Bottle" puzzle game, where the player moves balls of different colors between bottles. The goal is to sort the balls in each bottle by color, with the constraint that only balls of the same color can be moved together.

Purpose

The purpose of the FillUpTheBottle class is to:

1. **Initialize the Game:** Randomly populate the bottles with colored balls and ensure that the game setup is valid.
2. **Validate Moves:** Ensure that moves are only made when they adhere to the game's rules (e.g., moving a ball to an empty bottle or a bottle with the same color).
3. **Track Game State:** Monitor the state of the bottles to check if the game is complete.
4. **Track Moves:** Count the number of moves made during gameplay.

1.6.6 TowerOfHanoi Class

Introduction

The TowerOfHanoi class models the classic Tower of Hanoi puzzle game. The game involves moving a stack of tiles from one tower to another, following the rules that only one tile can be moved at a time and a larger tile cannot be placed on a smaller one.

Purpose

The purpose of the TowerOfHanoi class is to:

- **Initialize the Game:** Set up the towers with tiles in a specific starting configuration (largest tile at the bottom).
- **Validate Moves:** Ensure that the moves adhere to the game rules (e.g., no large tile can be placed on a smaller one).
- **Track Game Completion:** Check if the tiles have been moved to the target tower in the correct order.
- **Track Moves:** Keep count of the number of moves made during the game.

1.6.7 BinarySearchTree Class

Introduction

The BinarySearchTree (BST) class is an implementation of a self-balancing binary search tree, specifically an **AVL Tree**, which automatically balances itself during insertions. It ensures that the height of the left and right subtrees of any node differs by at most 1. This class provides methods to insert nodes, perform rotations to maintain balance, and check the properties of the tree such as whether it's balanced and whether it satisfies the binary search tree properties.

Purpose

The purpose of the BinarySearchTree class is to:

- **Maintain a Balanced Tree:** Automatically balance the tree during insertions to maintain efficient search, insertion, and deletion operations.
- **Insert Nodes:** Insert keys into the tree while ensuring it remains balanced by using rotations.
- **Balance Operations:** Perform left and right rotations to rebalance the tree when necessary.
- **Check Tree Properties:** Check if the tree is balanced and if it satisfies the binary search tree (BST) property.
- **Calculate Tree Height:** Calculate the height of the tree to assess balance and determine whether it meets the required structure.

1.7 Test Cases

Test cases are essential for validating the functionality, reliability, and performance of your code. Based on your report, here are some examples of test cases for each of the main components described:

1.7.1 General Test Case Structure

For each test case:

- **Test Case ID:** Unique identifier for the test case.
- **Test Description:** Briefly explain what the test will validate.
- **Input Data:** The data or conditions being tested.
- **Expected Result:** The expected behavior or outcome.

- **Actual Result:** What the system actually does (filled after execution).
- **Pass/Fail:** Result of the test.

1.7.2 Test Case for "Fill Up the Bottle" Game

Test Case 1: Valid Ball Movement

- **Test Description:** Ensure a ball can be moved to a valid target bottle.
- **Input Data:** A bottle containing a red ball on top; a target bottle that is empty.
- **Expected Result:** The red ball is successfully moved to the target bottle.
- **Actual Result:** Red Ball is easily filled in the bottle.

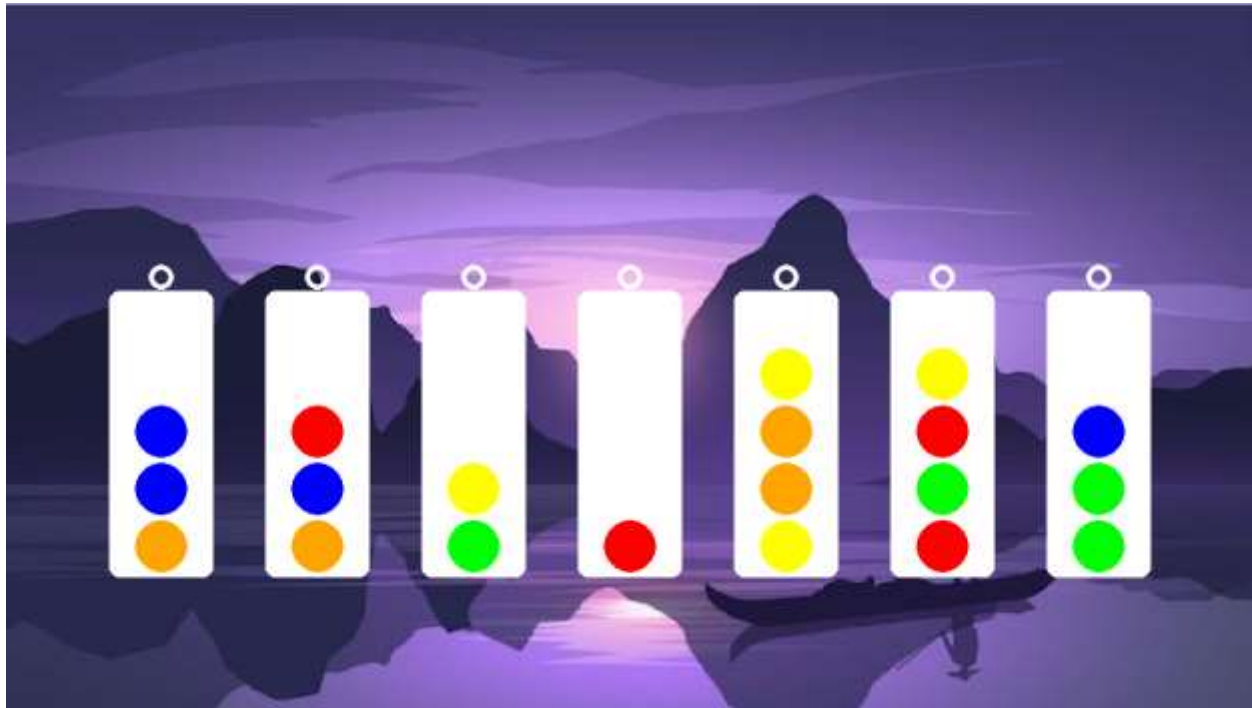


Figure 3 Test Case 1 (Valid Ball Movement)

- **Pass/Fail:** Passed.

Test Case 2: Invalid Ball Movement

- **Test Description:** Ensure a ball cannot be moved to a bottle with a different top color.
- **Input Data:** A bottle with a red ball on top; a target bottle with a blue ball on top.
- **Expected Result:** Movement is restricted, and a message (or visual cue) indicates the invalid move.
- **Actual Result:** Ball can't move to target bottle.

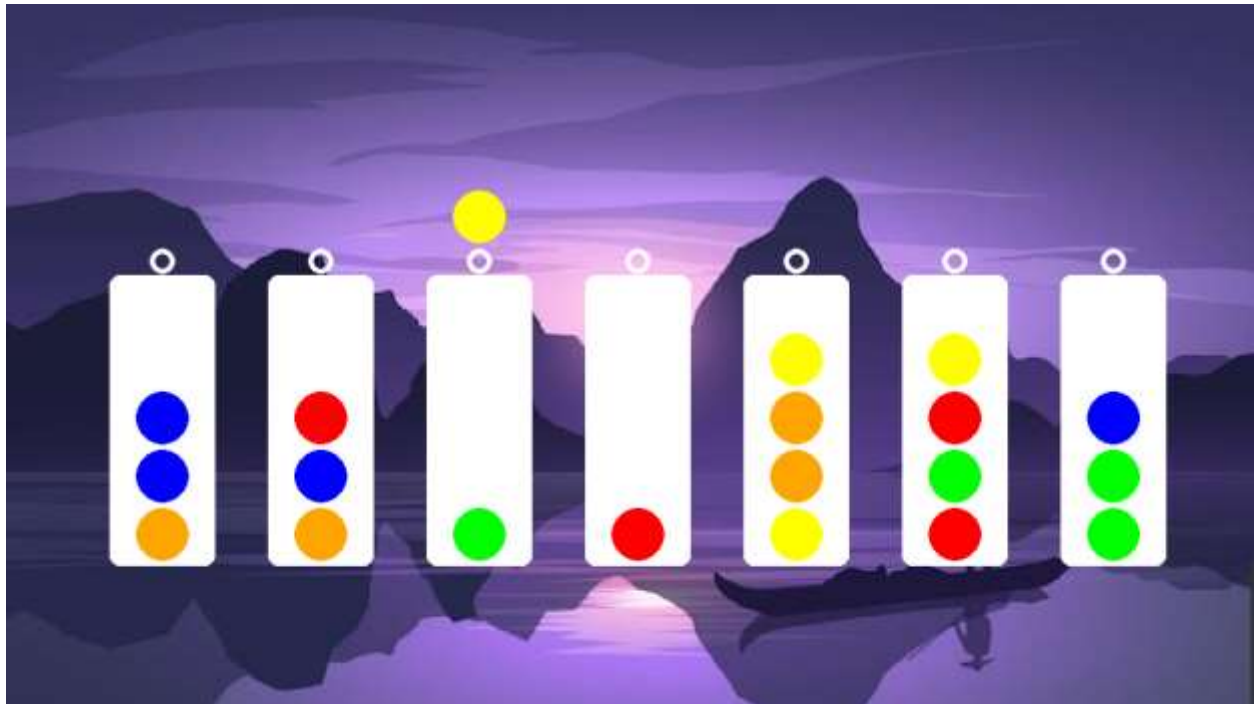


Figure 4 Test Case 2 (Invalid Ball Movement)

- **Pass/Fail:** Passed.

1.7.3 Test Case for "Tower of Hanoi" Game

Test Case 3: Disk Movement Rule

- **Test Description:** Test that a larger disk cannot be placed on a smaller disk.
- **Input Data:** Move a large disk to a rod where the top disk is smaller.
- **Expected Result:** The move is rejected, and the user is notified.
- **Actual Result:** Move is rejected and user is informed by a sound.



Figure 5 Test Case 3 (Movement of Tile)

- **Pass/Fail:** Passed.

1.7.4 Test Case for "Snake Evolution"

Test Case 4: Snake Collision with Itself

- **Test Description:** Verify that colliding with its body removes the collided segment and subsequent nodes.
- **Input Data:** Snake head collides with its body.
- **Expected Result:** The collided node and all subsequent nodes are removed from the snake.
- **Actual Result:** The body of snake is eaten and become smaller.



Figure 6 Test Case 4 (Collision)

- **Pass/Fail:** Passed.

1.7.5 Test Case for "Maze Solver"

Test Case 5: Invalid Maze

- **Test Description:** Ensure the algorithm handles unsolvable mazes gracefully.
- **Input Data:** A maze where the start and end points are disconnected.
- **Expected Result:** An error or message indicates that the maze is unsolvable.
- **Actual Result:** Error message does not show.
- **Pass/Fail:** Failed.

1.7.6 Test Case for "Traversal Tycoon"

Test Case 6: Pre-order Traversal

- **Test Description:** Verify the correctness of the pre-order traversal sequence.
- **Input Data:** A binary tree with nodes labeled [A, B, C, D, E].

- **Expected Result:** The sequence is [1, 8, 10, 3, 12, 7, 13, 14, 15, 11, 4, 2, 6, 9, 5].
- **Actual Result:** [1, 8, 7, 10, 13, 3, 14, 4, 2, 6, 5, 9, 15, 11, 12].

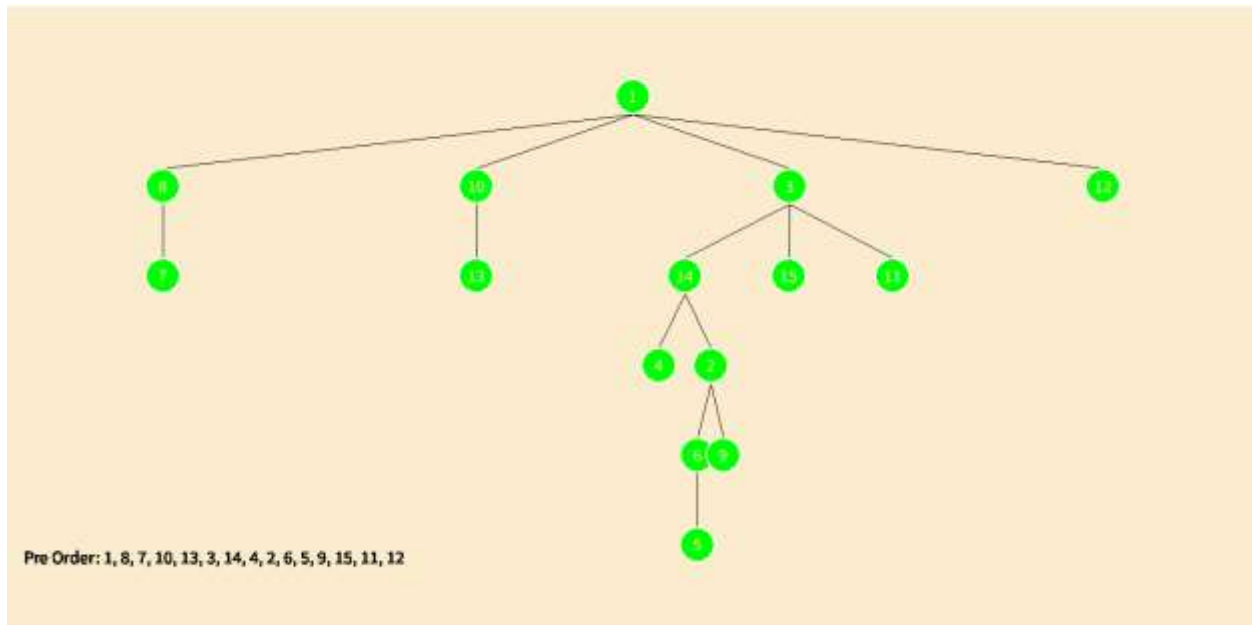


Figure 7 Test Case 6 (Pre Order)

- **Pass/Fail:** Passed.

1.7.7 Test Case for "Tic Tac Toe"

Test Case 7: Valid Move

- **Test Description:** Verify that a valid move updates the board correctly.
- **Input Data:** Player X marks position (0, 0) on a blank board.
- **Expected Result:** The board updates to:

```
X _ _
_ _ _
_ _ _
```

- **Actual Result:** It is proper placed.

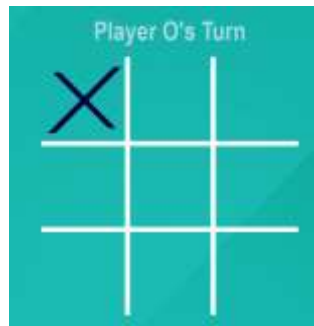


Figure 8 Test Case 7 (Placing X)

- **Pass/Fail:** Passed

1.7.8 Test Case for "Balanced Tree Formatter"

Test Case 8: Valid Node Insertion

- **Test Description:** Verify that a new node is correctly inserted into the Binary Search Tree (BST).
- **Input Data:** Insert a node with value 59 into the tree
- **Expected Result:** The tree updates with new node
- **Actual Result:** It is inserted properly.

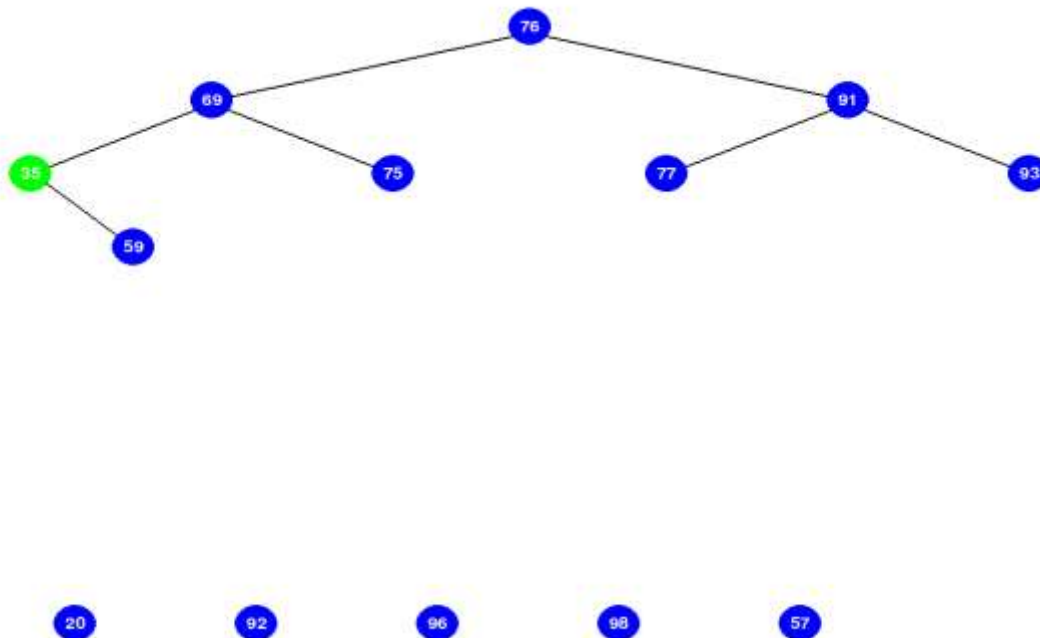


Figure 9 Test Case 8 (Insertion in tree)

- **Pass/Fail:** Passed

1.7.8 Test Cases for Quiz Component

Test Case 9: Correct Answer Submission

- **Test Description:** Ensure that a correct answer is identified and scored appropriately.
- **Input Data:** Select the correct option for a question.
- **Expected Result:** The score is incremented, and feedback is displayed.
- **Actual Result:** Same result came as expected.

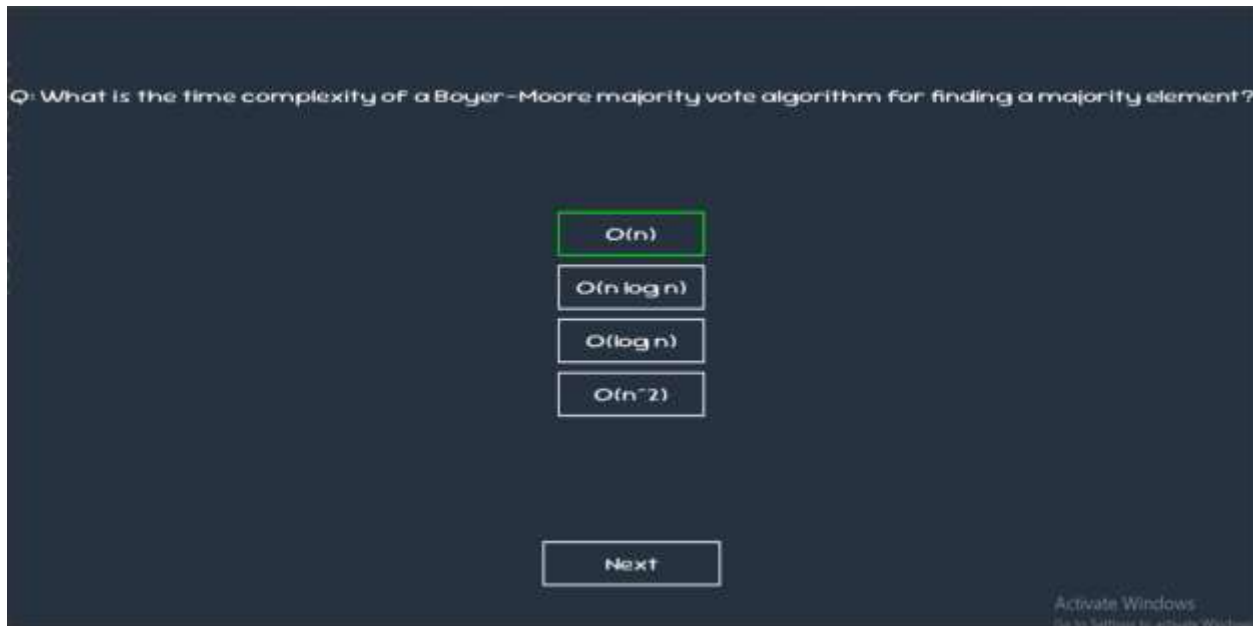


Figure 10 Test case 10 (Valid Answer)

- **Pass/Fail:** (Marked after testing).

1.8 Wireframes

1.8.1 Starting Menus

1.8.1.1 Main Page

The Main Page serves as the central hub of the project, providing users with easy navigation to key sections. It includes options to select a game, access the quiz menu for Data Structures or Algorithms, view the About Us section, or exit the application. The layout is clean, interactive, and user-friendly, ensuring smooth navigation.



Figure 11 Main Menu

1.8.1.2 Select Game Menu

Displays a variety of games with interactive icons, allowing users to choose and play games like puzzles, memory match, or tree-based challenges.

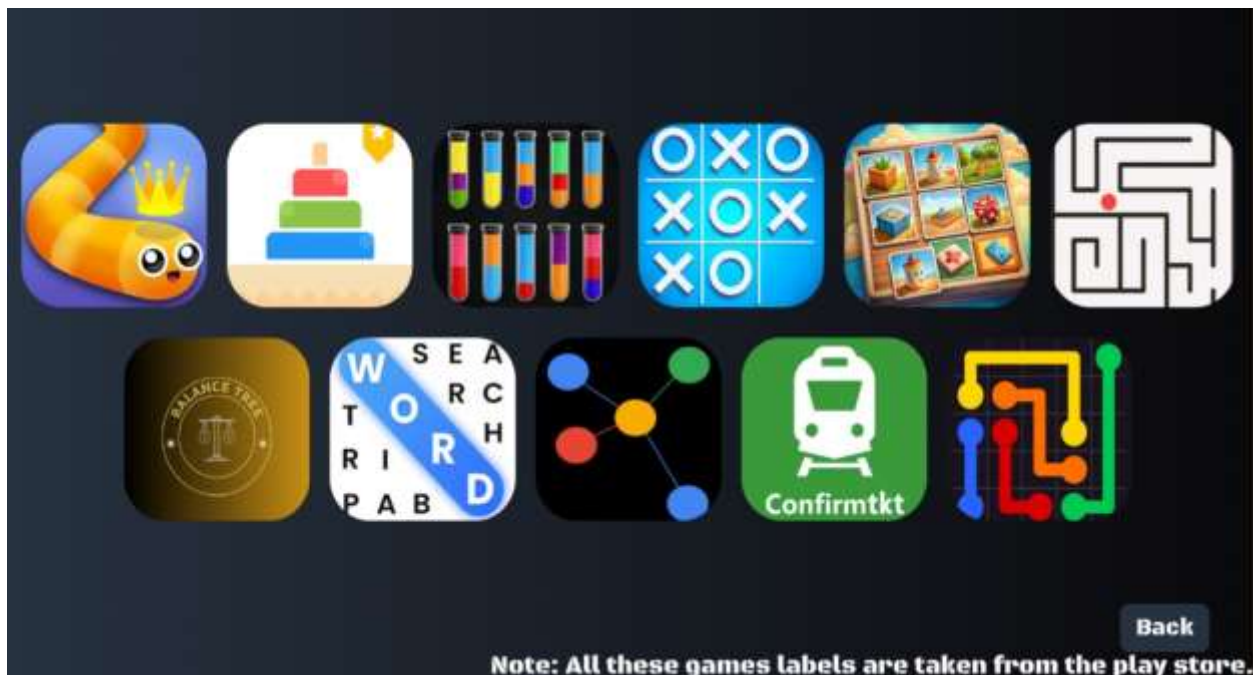


Figure 12 Game Menu

1.8.1.3 Quiz Menu

Offers two options for users to test their knowledge: one focusing on Data Structures and the other on Algorithms.



Figure 13 Quiz Menu

1.8.1.4 About Us Menu

Provides brief information about the project, its purpose, and the team involved.

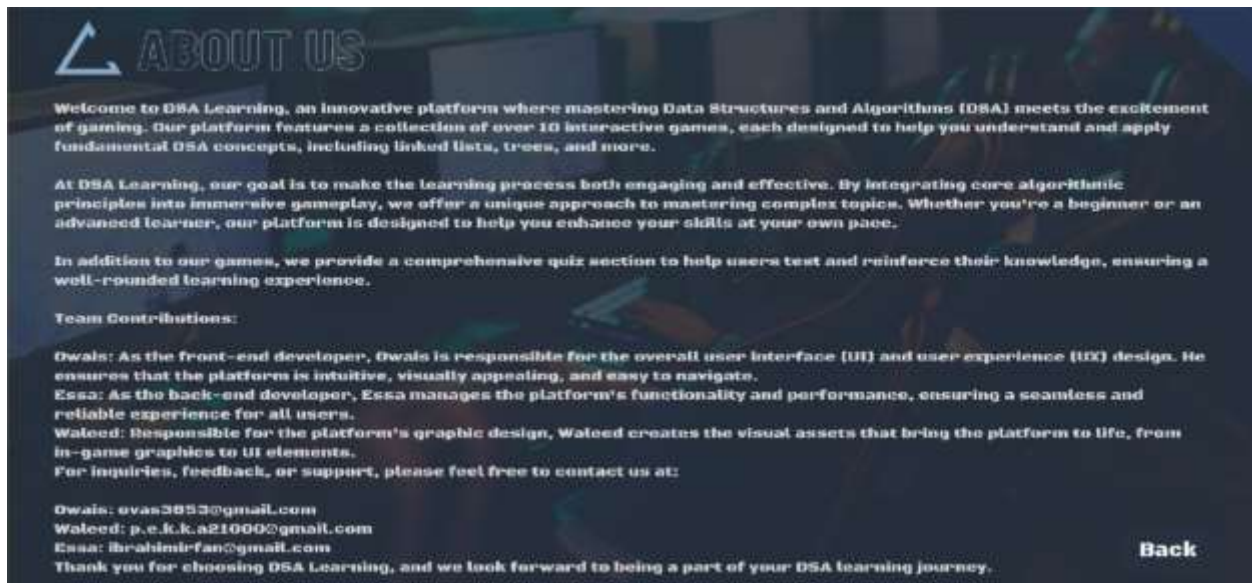


Figure 14 About Us Menu

1.8.2 Game Menus

Some game menus are already shown in test case portion so remaining game menus are shown below

1.8.2.1 Memory Match



Figure 15 Memory Match

1.8.2.2 Word Search

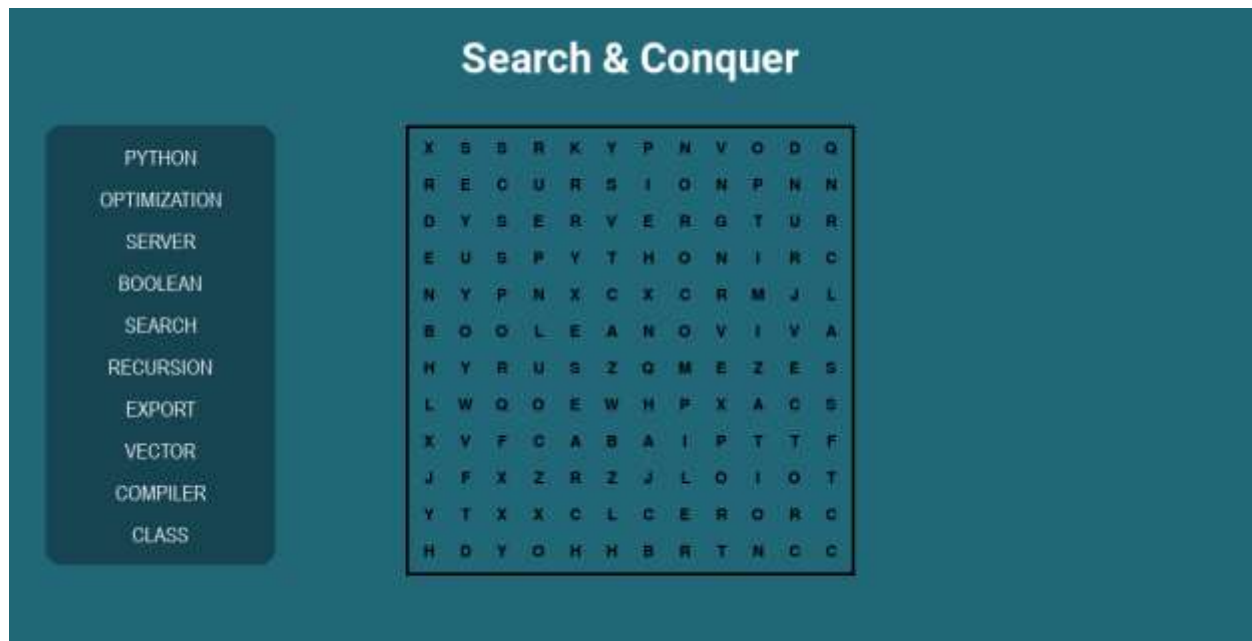


Figure 16 Word Search

1.8.2.3 One Way Out

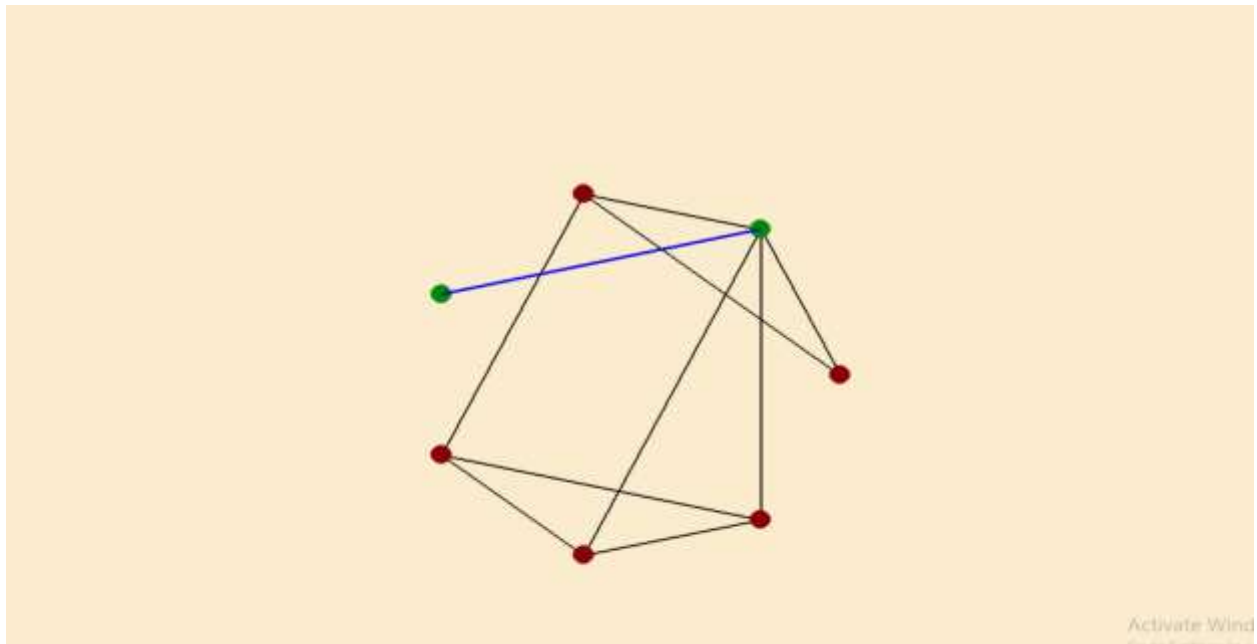


Figure 17 One Way Out

1.8.2.4 Maze Solver

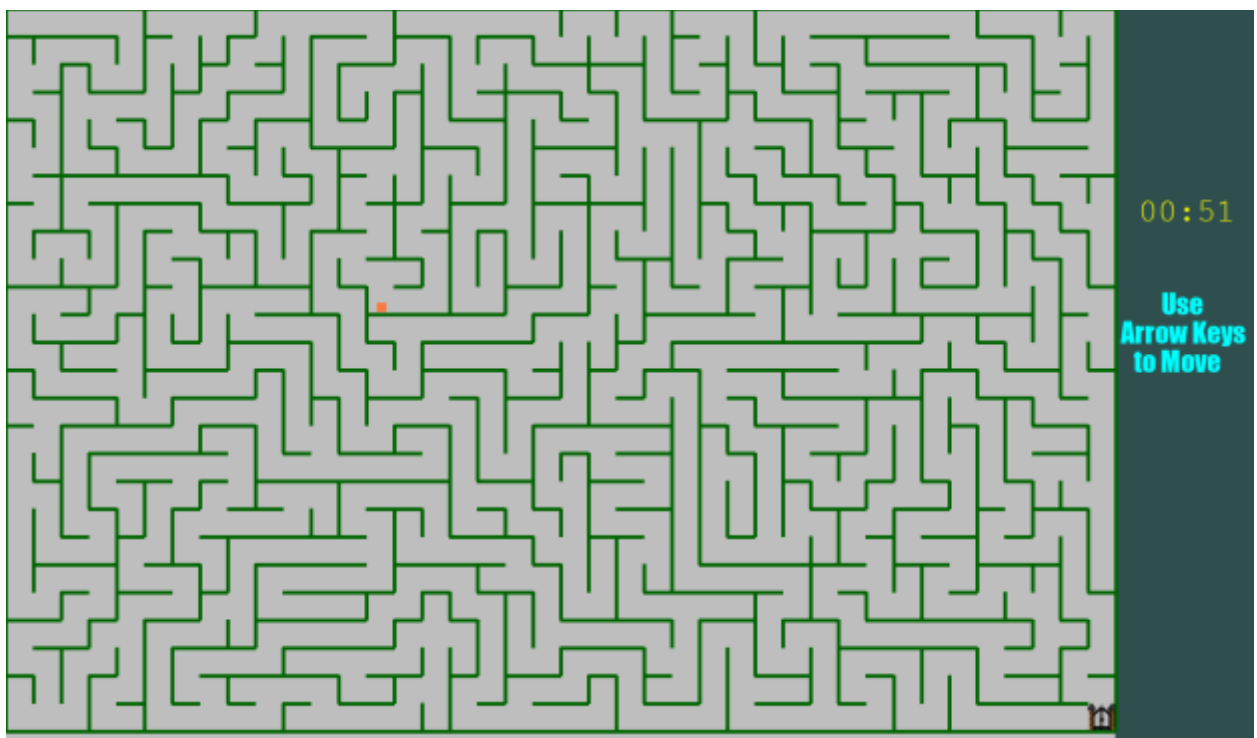


Figure 18 Maze Solver

1.8.3 Restart Menu

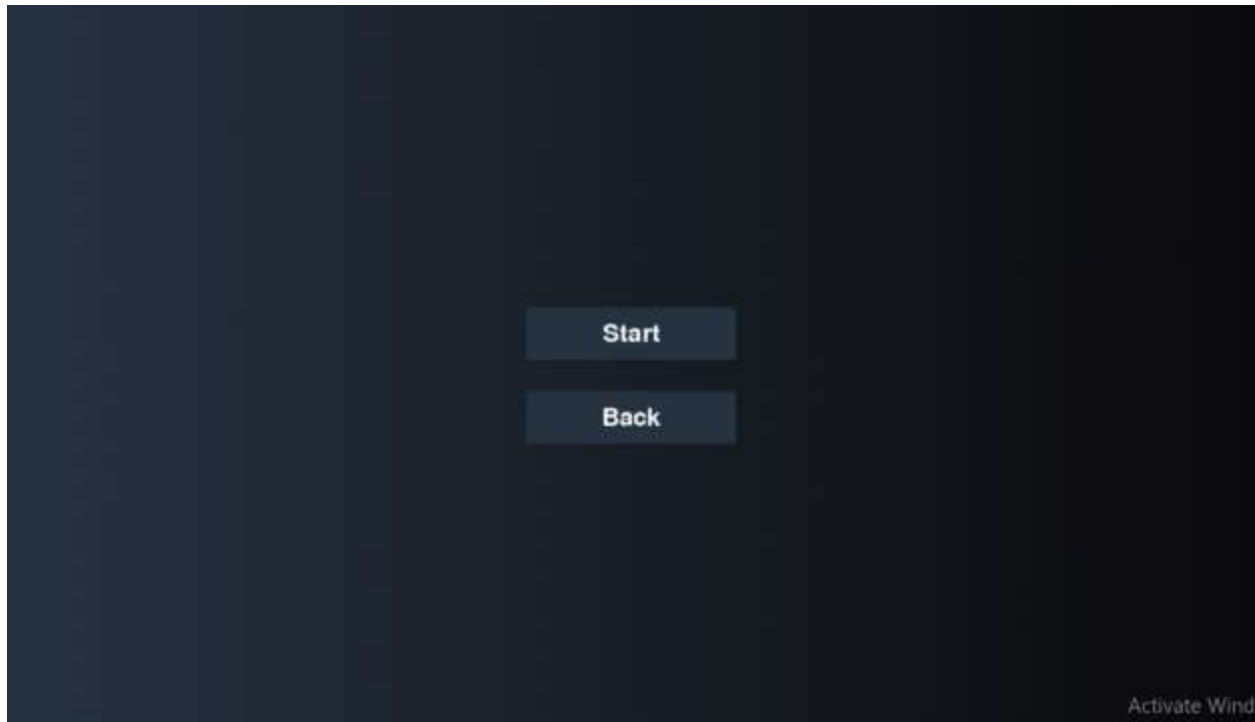


Figure 19 Restart Menu

1.9 Conclusion

This project successfully integrates various data structures and algorithms through engaging games like **Snake Evolution**, **Tower of Hanoi**, and **Word Search**. The use of classes such as Snake, TowerOfHanoi, and BinarySearchTree efficiently handles game logic and provides a fun way to learn concepts like tree traversal and recursion.

The modular design of the project ensures scalability and maintainability, while the self-balancing AVL tree implementation enhances performance. By combining interactive quizzes and games, the project offers an effective, hands-on approach to learning data structures and algorithms.

Overall, this project demonstrates how Python can be used to create both educational tools and entertaining games, providing users with a practical and engaging learning experience.

1.10 References

1. **Python Documentation.** (2023). Python Software Foundation. Retrieved from <https://docs.python.org/3/>
 - o This documentation provided valuable information on basic Python syntax and standard library usage, which helped implement various features in the project such as file handling, list manipulations, and algorithm implementations.

2. **Pygame Documentation.** (2023). Pygame Community. Retrieved from <https://www.pygame.org/docs/>
 - Pygame was used for handling the graphical interface of the games. The documentation was useful for understanding how to integrate images, handle user input, and render game objects on the screen.
3. **AVL Trees - Wikipedia.** (2023). Wikipedia. Retrieved from https://en.wikipedia.org/wiki/AVL_tree
 - The AVL Tree concept and self-balancing mechanism were implemented as part of the BinarySearchTree class. This reference was essential in understanding the theory behind balancing and rotations.